

Proyecto V6

16 de Abril del 2026

PROYECTO TUTURNO

APLICACIÓN WEB DE GESTIÓN DE CITAS ONLINE PARA UN SALÓN DE BELLEZA UNIPERSONAL



DESARROLLADO POR: JOSÉ MANUEL SÁNCHEZ ROSAL
FECHA: 09 DE ENERO DEL 2026

Índice

Índice.....	2
1- Introducción.....	3
2- Definición del Problema.....	3
2.1 Definición del Problema real.....	4
2.2 Problema Técnico.....	4
3. Objetivos.....	5
3.1 Objetivo Principal.....	5
3.2 Objetivos Específicos.....	5
4. Antecedentes.....	6
5. Recursos.....	7
5.1 Recursos Humanos y Conocimientos.....	7
5.2 Recursos Hardware.....	7
6. Análisis del Problema.....	8
6.1 Propuesta Software.....	8
6.1.1 Requisitos de Usuario.....	8
6.1.2 Requisitos Funcionales.....	9
6.1.3 Requisitos No Funcionales.....	9
7. Diseño de la solución software.....	10
7.1. Modelo relacional de la base de datos.....	10
7.2. Normalización de la base de datos.....	11
7.3. Paquetes o tecnologías.....	12
8. Diseño de la interfaz.....	13
8.1. Navegación y Encabezado.....	13
8.2. Catálogo de Servicios (Vista Cliente).....	13
8.3. Proceso de Reserva (Flujo Lineal).....	14
8.4. Paneles de Usuario (Perfiles).....	14
8.5. Adaptabilidad a Dispositivos (Checklist Mobile/PC).....	15
9. Pruebas.....	15
9.1. TuturnoApplicationTests (Verificación de Infraestructura).....	15
9.2. UsuarioServiceTest (Gestión de Identidad y Seguridad).....	15
9.3. CitaServiceTest (El Core de Negocio de Tuturno).....	16
9.4. Documento de entrada y salida.....	16

10. Conclusiones.....	17
10.1. Futuras Líneas de Mejora.....	17
11. Bibliografía.....	18
Documentación Oficial de Tecnologías.....	18
Libros y Manuales.....	19
Artículos y Recursos Web.....	19
Herramientas y Software Utilizado.....	19

1- Introducción

La situación particular que motiva este proyecto es **la dificultad que enfrenta un profesional autónomo para encontrar herramientas de gestión que se ajusten a su realidad**. La mayoría del software existente en el mercado está diseñado para grandes salones con múltiples empleados, resultando excesivamente complejo, costoso o con funcionalidades innecesarias para una gestión unipersonal.

Mi Proyecto Final Intermodular del **Ciclo Superior de Desarrollo de Aplicaciones Multiplataforma** consiste en el **desarrollo de una Aplicación Web de Gestión de Citas Online** para un Salón de Belleza y Peluquería unipersonal (autónoma).

El tiempo es Oro, por ello la aplicación busca digitalizar y automatizar el proceso de reserva, permitiendo a los clientes agendar citas 24/7 **liberando al profesional de la necesidad de atender llamadas o mensajes** constantes, optimizando su tiempo y flujo de trabajo diario (dentro o fuera de su jornada laboral).

2- Definición del Problema

Para abordar correctamente la solución, es necesario distinguir entre el problema operativo del negocio y los desafíos técnicos que conlleva su solución.

2.1 Definición del Problema real

El problema central es la ineficiencia en la gestión del tiempo y la comunicación del Profesional autónomo, lo que genera:

1. **Interrupciones constantes:** Las llamadas, los WhatsApps o mensajes directos vía redes sociales interrumpen continuamente el servicio al cliente presencial, disminuyendo la calidad de trabajo y la concentración.
2. **Gestión manual propensa a errores:** El uso de agendas o calendarios no automatizados conlleva riesgos de errores humanos, como dobles reservas (solapamiento), huecos perdidos por mala organización o citas olvidadas por falta de confirmación.
3. **Restricción horaria para el Cliente:** Actualmente, la captación de citas está limitada al momento en que el profesional puede responder, perdiendo oportunidades de reserva fuera del horario laboral o durante fines de semana.

2.2 Problema Técnico

Desde el **punto de vista del desarrollo de software**, la gestión manual actual **presenta las siguientes deficiencias técnicas**:

- **Ineficiencia en el cálculo de disponibilidad:** Solapamiento entre citas, la diferencia entre el tiempo estimado de duración del tratamiento y el tiempo real.
- **Falta de integridad y persistencia de datos:** Al no existir una base de datos centralizada, la información de los clientes al igual que su historial de citas está disperso y abierto a pérdidas físicas.
- **Falta de una plataforma centralizada de administración:** Actualmente, la gestión de los servicios ofrecidos, sus tarifas, la duración de cada tratamiento y los ajustes necesarios en la agenda dependen de métodos manuales (agenda física, notas, hojas sueltas, etc).

- **Ausencia de concurrencia:** El sistema actual no permite que múltiples usuarios consulten la disponibilidad al mismo tiempo sin riesgo de conflicto.
- **Solución mediante aplicación Multiplataforma.**
- **Realizar una Interfaz sencilla para solicitar cita o servicio en el salón de peluquería.**
- Impedir solicitar la misma cita (día y hora) por más de un cliente.

3. Objetivos

3.1 Objetivo Principal

Desarrollar una Aplicación Web funcional y robusta que automatice la reserva de citas, ajustando de manera dinámica la disponibilidad del calendario en función del tratamiento seleccionado, diseñada específicamente para las necesidades de una Profesional Autónoma.
Nombre App: "Tuturno"

3.2 Objetivos Específicos

1. **Automatización y Disponibilidad:** Permitir a los clientes la reserva, consulta, cancelación o modificación de citas a través de la aplicación **accesible las 24 horas del día**, 7 días a la semana, durante todo el año.
2. **Lógica de agenda Dinámica:** Implementar en el servidor la capacidad lógica de calcular y bloquear automáticamente tramos horarios correctos en el calendario, considerando la duración del tratamiento y los tiempos de preparación y limpieza necesarios.
3. **Gestión administrativa Centralizada:** Proporcionar un papel de administración seguro e intuitivo donde el profesional pueda gestionar sus servicios y tarifas sin necesidad de conocimientos técnicos avanzados (**CRUD servicios y citas bajo el Rol de Administrador**).

4. **Escalabilidad y mantenibilidad:** Desarrollar el sistema bajo una arquitectura de Software Profesional que asegure un código limpio, mantenible y escalables a futuras funcionalidades del negocio.
5. **Gestión de Usuarios:** la aplicación permitirá el alta y baja de clientes.

4. Antecedentes

El proyecto surge tras analizar el mercado de software comercial de gestión de salones y detectar una brecha entre la oferta y las necesidades reales de una microempresa.

Análisis de Soluciones Existentes: El mercado ofrece soluciones robustas y reconocidas como Booksy, Fresha o Koibox. A continuación se detallan algunas de sus características:

1. **Booksy:** Es una plataforma muy popular y completa que permite reservas online 24/7, gestión de agenda, clientes, pagos y marketing. Muy útil si quieres visibilidad (mercado de usuarios muy grande) y abarcar múltiples servicios en un solo lugar.
2. **Fresha:** Sistema de reservas orientado a salones de belleza y bienestar, con interfaz limpia y fácil de usar.
3. **Koibox:** Es un software de gestión integral para salones de peluquería y belleza, incluye agenda, gestión de clientes, inventario, informes. También herramientas de administración interna y reservas online

Ahora veremos los puntos débiles de los mismos, punto crucial para la decisión de que estas apps no encajan en el perfil del Autónomo Unipersonal:

- A. **Exceso de funcionalidades innecesarias:** todas incluyen módulos avanzados (stock, marketing, analíticas complejas, gestión multiempleo) que un profesional autónomo no necesita, haciendo la interfaz más compleja y menos intuitiva.
- B. **Costes recurrentes:** Muchos servicios requieren de una suscripción mensual, pagos adicionales o comisiones por reserva, lo cual incrementa los gastos fijos del autónomo.
- C. **Dependencia de plataformas externas:** Los datos, agenda y disponibilidad dependen de servidores y lógicas externas, limitando la personalización. Además, los clientes pueden ver otros negocios, creando competencia.
- D. **Falta total de personalización:** Estas apps presentan estructuras rígidas para configurar servicios, duración, descansos o disponibilidad.

- E. **Curva de aprendizaje innecesaria:** Su complejidad está pensada para equipos grandes, obliga al profesional a aprender funciones que nunca usará.

Por lo tanto, este proyecto busca **replicar en parte la funcionalidad central** de reserva de estas grandes plataformas pero en una solución hecha a medida, **gratuita** (sin costes de licencia) y **optimizada para** la agilidad de **un solo Usuario**.

5. Recursos

5.1 Recursos Humanos y Conocimientos

- **Coordinador del Proyecto:** Pedro Rafael García Velasco (Tutor y profesor de asignaturas clave).
- **Desarrollador Principal:** José Manuel Sánchez Rosal (Estudiante de Ciclo Superior DAM)
- **Asesor y colaborador:** José Julio Soldado Carmona (ex-alumno Ciclo Superior DAM)
- **Conocimientos Técnicos Requeridos:** para la ejecución del Proyecto se aplicarán conocimientos avanzados de desarrollo web Full Stack, seleccionando las herramientas más adecuadas durante la fase de análisis:
 - Backend: Desarrollo de API REST segura y robusta utilizando lenguajes de programación de servidor y frameworks de desarrollo ágil.
 - Persistencia: Diseño y gestión de Bases de Datos Relacionales y normalización de datos para el manejo eficiente de Usuarios, Servicios y Transacciones.
 - Frontend: Desarrollo de Interfaces Web dinámicas y responsivas y accesibles basadas en estándares web actuales.
 - DevOps: Control de versiones distribuidos y despliegue en entornos de servidor en la nube.

5.2 Recursos Hardware

- Portátil Medion Akoya Intel CORE i5 (8ª GEN) con 16 gb de RAM.
- Monitor Samsung de 27" resolución Full HD con panel PLS.

6. Análisis del Problema

Vamos a desglosar la definición de nuestro problema para sacar a luz nuestros requisitos:

6.1 Propuesta Software

Los requisitos que debe cumplir nuestra app, se dividen en tres grupos:

6.1.1 Requisitos de Usuario

Tenemos dos tipos de Rol de usuario, el **usuario normal** y el **usuario administrador**. A través de la página, en la pestaña "zona clientes", se podrá acceder al registro de usuarios mediante un formulario. A continuación se muestran los requisitos de Usuario de cada uno:

- Un Usuario debe poder registrarse.
- Usuario y Administrador podrán loguearse.
- Usuario y Administrador podrán acceder al calendario.
- El usuario podrá visualizar los días y huecos disponibles para reservar un servicio (las citas ocupadas serán anónimas, sin poder ver el cliente que la tiene ocupada).
- El administrador podrá ver los días y huecos disponibles para visualizar todas y cada una de las citas de sus clientes.
- El Usuario podrá seleccionar fecha y hora y servicio requerido para reservar una cita.
- Usuario y Administrador podrán modificar la cita.
- Usuario y Administrador podrán eliminar la cita.
- El Usuario podrá darse de baja como usuario de la página y dejará de tener acceso al calendario y servicios.
- El administrador podrá gestionar el catálogo de servicios, teniendo capacidad para crear nuevos tratamientos, modificar sus datos o eliminarlos.
- El usuario recibirá una confirmación visual de confirmación de reserva.

6.1.2 Requisitos Funcionales

- Se podrá enviar un formulario con los datos del cliente al servidor para su registro.
- Cliente y administrador, al enviar sus datos de inicio de sesión al servidor a través de un formulario, recibirá ciertos permisos en la página .
- Cliente y administrador una vez logueados, podrán acceder al calendario de citas.
- En el calendario, el usuario podrá verificar los huecos disponibles.
- En el calendario, el administrador podrá visualizar las citas ocupadas por cada cliente.
- El usuario podrá seleccionar fecha, hora y servicio seleccionado para reservar su cita, la información será enviada a modo de formulario.
- Usuario y administrador podrán modificar citas, a través del formulario.
- Usuario y administrador podrán acceder al calendario y eliminar la cita a través del formulario.
- El usuario podrá enviar un formulario para darse de baja como cliente para ser eliminado.
- El administrador podrá dar de alta, editar y eliminar servicios a través de un formulario.
- Cálculo de tiempos, el sistema calculará la disponibilidad de huecos sumando automáticamente el tiempo de "limpieza y preparación".

6.1.3 Requisitos No Funcionales

Adaptabilidad (Diseño Responsivo): La aplicación debe visualizarse y funcionar correctamente tanto en ordenadores como en dispositivos móviles, ya que los clientes reservarán principalmente desde el teléfono.

Integridad y persistencia de Datos (Cero Solapamientos): El sistema debe garantizar técnicamente que nunca se puedan guardar dos citas en el mismo tramo horario. Si dos usuarios intentan reservar el mismo hueco al mismo tiempo, uno debe recibir un error amigable.

Por otro lado, el administrador tendrá la seguridad de tener a salvo la información a través de la bbdd.

Seguridad de Acceso (Autenticación): El acceso a la zona de administración y perfil de usuario debe requerir usuario y contraseña.

Facilidad de Uso (Usabilidad): El proceso de reserva debe ser lineal y sencillo: Seleccionar Servicio -> Seleccionar Fecha/Hora -> Confirmar. No debe requerir conocimientos técnicos por parte del cliente.

7. Diseño de la solución software

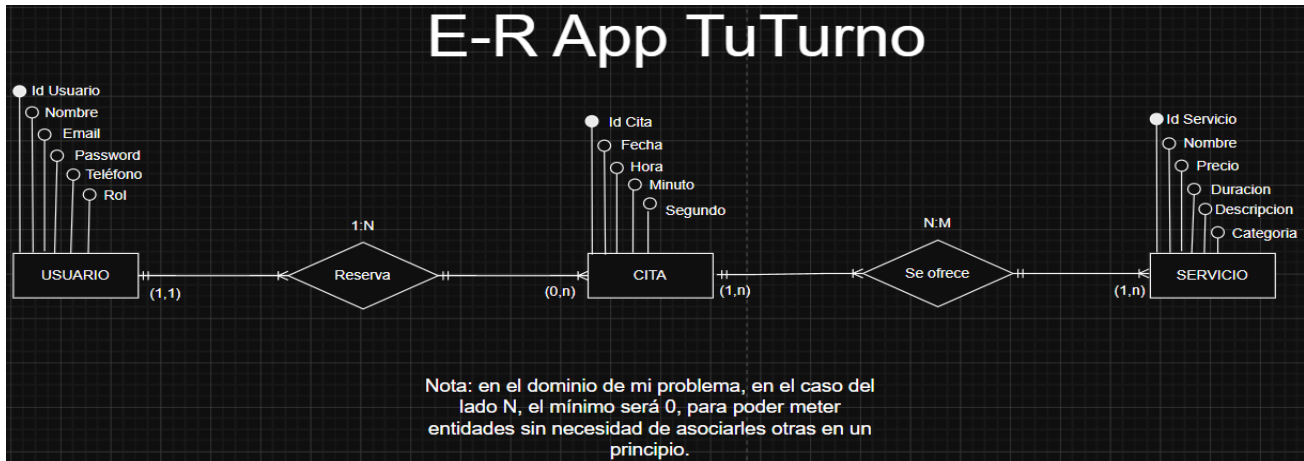
El sistema "Tuturno" se ha diseñado bajo una arquitectura cliente-servidor desacoplada, lo que permite una separación total entre la interfaz de usuario y la lógica de negocio. El **Backend** está construido como una API RESTful desarrollada en Java con el framework Spring Boot, siguiendo un patrón de diseño multicapa que separa controladores (exposición de servicios), lógica de negocio (capa de servicios) y persistencia (capa de repositorios). Por su parte, el **Frontend** se ha implementado con una interfaz web modular basada en estándares modernos (HTML5, CSS3 y Vanilla JavaScript), garantizando una comunicación asíncrona eficiente mediante peticiones HTTP.

7.1. Modelo relacional de la base de datos

El sistema utiliza PostgreSQL como base de datos relacional, gestionada mediante JPA/Hibernate. El esquema se compone de las siguientes tablas:

- **Tabla usuarios:** Almacena la información de acceso y contacto. Atributos: **id** (PK), **nombre**, **email**, **password**, **telefono** y **rol** (almacenado como String mediante el Enum **TipoRol**).
- **Tabla servicios:** Contiene el catálogo de tratamientos. Atributos: **id** (PK), **nombre**, **descripcion**, **precio**, **duracion_minutos** y **categoria**.
- **Tabla citas:** Registra las reservas de tiempo. Atributos: **id** (PK), **fecha**, **hora**, **minutos**, **segundos** y **usuario_id** (FK).

- **Tabla intermedia Cita_Servicio:** Tabla necesaria para resolver la relación de muchos a muchos (@ManyToMany), vinculando múltiples servicios a una sola cita y viceversa. Atributos: `id_cita` (FK) e `id_servicio` (FK):



7.2. Normalización de la base de datos

El diseño garantiza la integridad y eficiencia de los datos cumpliendo las tres primeras formas normales:

- **Primera Forma Normal (1FN):** Se garantiza que todos los atributos sean atómicos y no existan grupos repetitivos. El ejemplo principal es la tabla intermedia `Cita_Servicio`, que desglosa la relación entre citas y tratamientos, evitando listas de valores en una sola columna.
- **Segunda Forma Normal (2FN):** Se cumple al estar en 1FN y asegurar que todos los atributos no clave dependan funcionalmente de forma completa de la clave primaria (PK). Dado que todas las tablas utilizan un `id` autoincremental como PK única, no existen dependencias parciales.
- **Tercera Forma Normal (3FN):** Se cumple al no existir dependencias transitivas entre atributos no clave. Por ejemplo, en la tabla `usuarios`, el teléfono o el email dependen directamente del `id` del usuario y no de otros atributos no clave. Igualmente, los detalles

de un servicio (precio, duración) dependen exclusivamente de su identificador único de servicio.

7.3. Paquetes o tecnologías

Para el desarrollo del proyecto "Tuturno", se han seleccionado las siguientes tecnologías, frameworks y entornos de desarrollo, organizados según su área de aplicación:

Backend (Servidor y Base de Datos)

- **Java:** Lenguaje de programación principal estructurado bajo el paradigma de orientación a objetos.
- **Spring Boot:** Framework principal utilizado para la creación de la API RESTful.
- **Spring Data JPA / Hibernate:** Herramientas de mapeo objeto-relacional (ORM) para la comunicación y persistencia de datos con la base de datos.
- **Lombok:** Librería utilizada para reducir el código repetitivo en las clases Java (como *getters*, *setters* y constructores). * **PostgreSQL:** Sistema de gestión de bases de datos relacional para almacenar la información de usuarios, citas y servicios. * **IntelliJ IDEA:** Entorno de Desarrollo Integrado (IDE) principal utilizado para la programación de la lógica de servidor.

Frontend (Interfaz de Usuario) * HTML5 y CSS3:

Lenguajes estándar para la estructura semántica y el diseño visual y responsivo de la aplicación. * **Vanilla JavaScript:** Utilizado para la lógica del lado del cliente, manipulación del DOM y peticiones asíncronas HTTP a la API, sin depender de frameworks adicionales. * **Visual Studio Code:** Editor de código ligero utilizado para el desarrollo de la interfaz web.

Pruebas (Testing)

- **Postman:** Plataforma y cliente API utilizado de forma intensiva para diseñar, probar y validar el correcto funcionamiento de los distintos *endpoints* del Backend antes de su integración con el Frontend.

- **Control de Versiones** * **Git**: Sistema de control de versiones distribuido para el seguimiento de los cambios en el código fuente. * **GitHub**: Plataforma en la nube utilizada para alojar el repositorio central del proyecto y respaldar el código.

8. Diseño de la interfaz

Introducción y Principios de Diseño

El diseño de la interfaz (UI) de "Tuturno" se ha centrado en dos pilares fundamentales: la sencillez y la adaptabilidad. El objetivo es que el cliente pueda realizar una reserva en menos de un minuto, sin necesidad de formación previa ni ayuda técnica. Siguiendo los principios de usabilidad definidos en los requisitos no funcionales, el diseño es **completamente adaptativo (Responsive Design)**. Esto garantiza una experiencia de usuario (UX) óptima e intuitiva en cualquier dispositivo: **ordenadores personales (PC), tablets y teléfonos móviles**.

8.1. Navegación y Encabezado

El encabezado (Header) se mantiene fijo en la parte superior para proporcionar acceso rápido a la navegación. Incluye:

- **Logotipo de Marca**: Refuerza la identidad del salón de belleza.
- **Menú de Navegación Principal**: En resoluciones de PC y tablet, se muestran enlaces directos como "Inicio", "Servicios", "Zona Clientes" y "Contacto". En dispositivos móviles, este menú se contrae dentro de un icono de "hamburguesa" (☰) para maximizar el espacio de contenido.
- **Botón de Acción**: Un acceso directo visible para "Reservar Cita" o "Perfil/Salir" (según el estado de login).

8.2. Catálogo de Servicios (Vista Cliente)

Diseñado como una galería visual, el catálogo presenta los servicios organizados por tarjetas. Cada tarjeta incluye:

- **Icono Descriptivo:** Facilita la identificación visual del tratamiento.
- **Título y Descripción Breve:** Explica el servicio.
- **Detalles Clave:** Duración estimada (en minutos) y Precio, mapeados directamente desde la entidad [Servicio](#).
- **Selector de Reserva:** Un checkbox que permite al usuario añadir el servicio a su cita.

8.3. Proceso de Reserva (Flujo Lineal)

El flujo de reserva se ha simplificado al máximo para garantizar la usabilidad:

1. **Selección:** El usuario elige sus servicios del catálogo.
2. **Calendario Dinámico:** Se muestra un calendario interactivo que, basándose en la lógica del backend ([Servicio](#), [Cita](#) y tiempos de limpieza), bloquea automáticamente los huecos no disponibles. El usuario selecciona día y hora.
3. **Confirmación:** Una pantalla de resumen permite verificar la cita (fecha, hora, servicios) antes de enviarla. Una vez confirmada, el usuario ve una notificación visual de éxito.

8.4. Paneles de Usuario (Perfiles)

- **Zona Clientes:** Una vista limpia donde el usuario puede gestionar sus datos personales ([Usuario](#)) y visualizar una lista de sus "Próximas Citas" ([Cita](#)), con opciones visibles para "Modificar" (lápiz) o "Cancelar" (papelera).
- **Panel de Administración (Jefatura):** Diseñado para la eficiencia, el administrador accede a una vista avanzada del calendario donde puede visualizar todas las citas ocupadas, identificando a los clientes (con el rol de ADMIN). Desde este panel, se pueden realizar CRUD completos sobre los servicios y las citas de la agenda general.

8.5. Adaptabilidad a Dispositivos (Checklist Mobile/PC)

El diseño se adapta automáticamente de la siguiente manera:

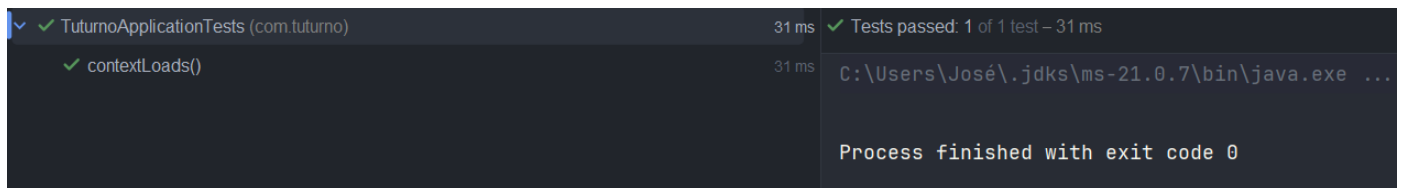
- **PC/Tablet:** El diseño se expande horizontalmente, aprovechando el ancho de pantalla.
- **Móvil:** Las tarjetas de servicios se compactan y el menú se oculta para una navegación táctil fluida.

9. Pruebas

El apartado de pruebas del proyecto certifica la solidez técnica de la aplicación "TuTurno" mediante la validación automatizada de su infraestructura y lógica de negocio principal. A través de diferentes suites, se audita el correcto arranque del servidor, la seguridad estricta en el cifrado de contraseñas y la gestión integral de identidades. Además, se somete a estrés el núcleo de la aplicación, verificando minuciosamente el sistema de permisos por roles, la disponibilidad de los horarios y la prevención de solapamientos en las citas.

9.1. TuturnoApplicationTests (Verificación de Infraestructura)

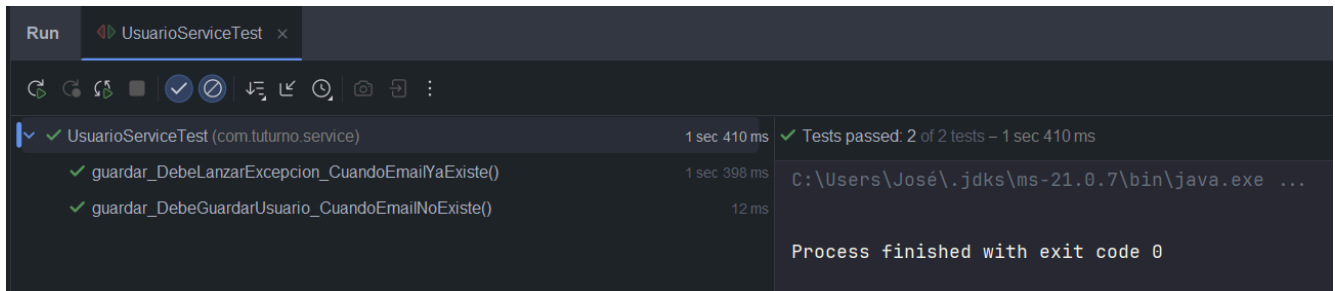
- Verifica que el contexto de Spring Boot y la inyección de dependencias se inicializan correctamente sin errores críticos de arranque:



```
✓ TuturnoApplicationTests (com.tuturno) 31 ms ✓ Tests passed: 1 of 1 test – 31 ms
  ✓ contextLoads() 31 ms C:\Users\José\.jdk\ms-21.0.7\bin\java.exe ...
  Process finished with exit code 0
```

9.2. UsuarioServiceTest (Gestión de Identidad y Seguridad)

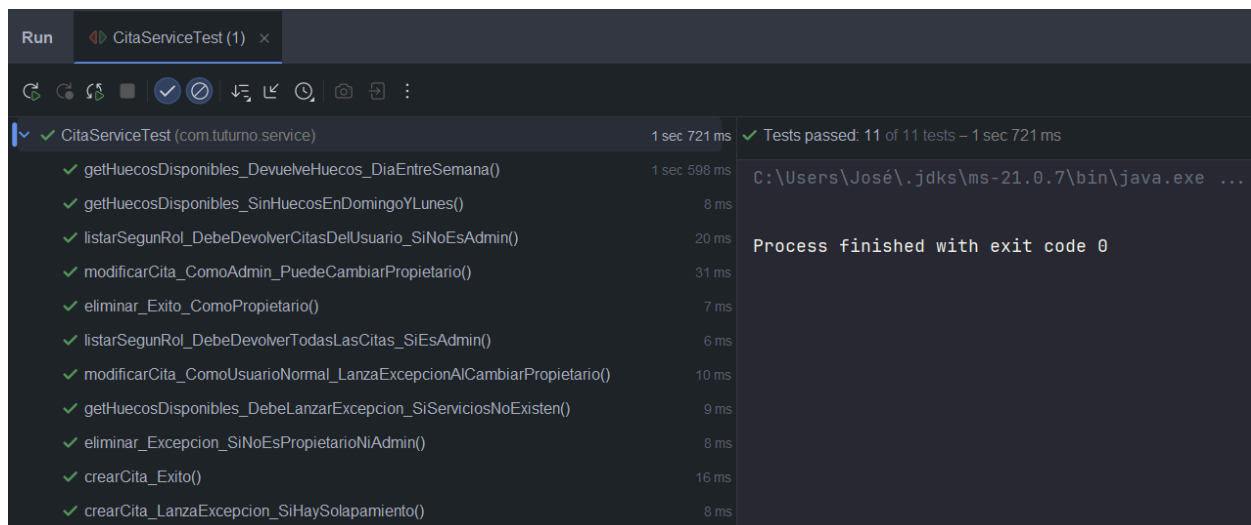
Valida que las contraseñas se encriptan (hash) obligatoriamente antes de almacenarse y bloquea el registro si el correo electrónico ya existe:



```
Run  UsuarioServiceTest x
✓ UsuarioServiceTest (com.tuturno.service) 1 sec 410 ms ✓ Tests passed: 2 of 2 tests – 1 sec 410 ms
  ✓ guardar_DebeLanzarExcepcion_CuandoEmailYaExiste() 1 sec 398 ms C:\Users\José\.jdk\ms-21.0.7\bin\java.exe ...
  ✓ guardar_DebeGuardarUsuario_CuandoEmailNoExiste() 12 ms
  Process finished with exit code 0
```

9.3. CitaServiceTest (El Core de Negocio de Tuturno)

Audita el control de permisos según el rol (Admin vs. Usuario común), el cálculo matemático de huecos disponibles (respetando días inhábiles y tiempos de preparación) y el bloqueo estricto de reservas para evitar solapamientos:



```
Run  CitaServiceTest (1) x
✓ CitaServiceTest (com.tuturno.service) 1 sec 721 ms ✓ Tests passed: 11 of 11 tests – 1 sec 721 ms
  ✓ getHuecosDisponibles_DevuelveHuecos_DiaEntreSemana() 1 sec 598 ms C:\Users\José\.jdk\ms-21.0.7\bin\java.exe ...
  ✓ getHuecosDisponibles_SinHuecosEnDomingoYLunes() 8 ms
  ✓ listarSegunRol_DebeDevolverCitasDelUsuario_SiNoEsAdmin() 20 ms
  ✓ modificarCita_ComoAdmin_PuedeCambiarPropietario() 31 ms
  ✓ eliminar_Exito_ComoPropietario() 7 ms
  ✓ listarSegunRol_DebeDevolverTodasLasCitas_SiEsAdmin() 6 ms
  ✓ modificarCita_ComoUsuarioNormal_LanzaExcepcionAlCambiarPropietario() 10 ms
  ✓ getHuecosDisponibles_DebeLanzarExcepcion_SiServiciosNoExisten() 9 ms
  ✓ eliminar_Excepcion_SiNoEsPropietarioNiAdmin() 8 ms
  ✓ crearCita_Exito() 16 ms
  ✓ crearCita_LanzaExcepcion_SiHaySolapamiento() 8 ms
  Process finished with exit code 0
```


9.4. Documento de entrada y salida

La tabla presenta una matriz estructurada con seis casos de prueba específicos (del CP-01 al CP-06) que validan el comportamiento del sistema ante diferentes eventos. En ella se definen claramente los datos de entrada y las salidas esperadas para situaciones cotidianas, como el cálculo de disponibilidad en días laborables frente a días de cierre (domingos y lunes). Asimismo, detalla pruebas de seguridad crítica, como la prevención de reservas simultáneas para evitar solapamientos y la restricción de permisos al intentar modificar citas de terceros sin rol de administrador, concluyendo que todos los escenarios se han ejecutado con éxito:

ID Prueba	Descripción	Datos de Entrada (Input)	Salida Esperada (Expected Output)	Resultado
CP-01	Cálculo de huecos en día laboral	Fecha: Jueves laborable Servicio: 45 minutos	Lista de franjas horarias con tamaño 45 mins separadas por tiempos de limpieza.	✓ Éxito (Build Success)
CP-02	Cálculo de huecos en día de cierre	Fecha: Lunes / Domingo Servicio: Cualquier duración	Lista vacía [] (Sin disponibilidad).	✓ Éxito (Build Success)
CP-03	Crear cita correctamente	Hora: 10:00 - 10:45 Estado: Sin solapes	Cita creada exitosamente y guardada en el simulador de Base de Datos.	✓ Éxito (Build Success)
CP-04	Prevención de Solapamiento	Hora: 10:00 - 10:45 Estado previo: Ya hay una cita a las 10:15	Error del sistema: <code>IllegalArgumentException</code> ("El servicio se solapa con una cita existente").	✓ Éxito (Build Success)
CP-05	Fallo por manipulación de ID	Usuario A intenta modificar una cita que pertenece a Usuario B.	Error del sistema: <code>SecurityException</code> o Acceso Denegado (403 Forbidden).	✓ Éxito (Build Success)
CP-06	Acceso Administrativo	Usuario con rol ADMIN intenta modificar una cita del Usuario B.	Cita modificada exitosamente (Privilegio superior aplicado).	✓ Éxito (Build Success)

10. Conclusiones

Resumen de Logros y Solución de Problemas El desarrollo del proyecto "Tuturno" ha concluido con éxito, cumpliendo con el objetivo principal de crear una Aplicación Web robusta y funcional que automatiza la gestión de citas online para un salón de belleza unipersonal. La solución implementada resuelve eficazmente el problema operativo de las interrupciones constantes, la gestión manual propensa a errores y la restricción horaria, proporcionando una herramienta profesional que digitaliza y optimiza el flujo de trabajo de un autónomo.

Solidez Técnica y Escalabilidad Técnicamente, la arquitectura cliente-servidor desacoplada (Backend API REST en Spring Boot / Frontend Vanilla JS) ha demostrado ser una elección acertada, garantizando un código limpio, mantenible y escalable. El esquema relacional de la base de datos, totalmente normalizado (1FN, 2FN, 3FN) como se detalla en el Apartado 7, asegura la integridad y persistencia de los datos, eliminando riesgos de redundancia y solapamientos de citas.

Experiencia de Usuario y Adaptabilidad La interfaz, diseñada bajo principios de usabilidad y adaptabilidad, ofrece una experiencia de usuario excelente tanto para el cliente como para el administrador. El diseño completamente responsive asegura la accesibilidad total desde **teléfonos móviles, tablets y PC**, cumpliendo con la premisa de que los clientes reservarán principalmente desde el teléfono.

10.1. Futuras Líneas de Mejora

- **Acceso directo mediante Código QR:** Tras el despliegue del sistema en un entorno de producción real, se generará un código QR físico para colocar en el mostrador del salón. Esto permitirá a los clientes presenciales escanearlo e instalar la aplicación en sus dispositivos móviles de forma instantánea, fomentando la transición hacia la reserva digital.
- **Integración de pasarela de pago:** Se añadirá un sistema de cobro online seguro dentro del flujo de reserva de la aplicación. Esta funcionalidad permitirá solicitar el pago de una fianza o el importe total por adelantado, garantizando el compromiso de asistencia del cliente y reduciendo significativamente las citas fallidas (no-shows).
- **Módulo de contabilidad automatizada:** Se desarrollará una nueva sección en el panel de administración enfocada en la gestión financiera del negocio. Este módulo registrará los ingresos de los servicios prestados y generará informes o resúmenes periódicos, facilitando al profesional autónomo el control de su facturación y la preparación de impuestos.

11. Bibliografía

Nota: La bibliografía citada a continuación corresponde a la documentación de las tecnologías candidatas en la fase de propuesta inicial. Estas referencias podrán actualizarse:

Documentación Oficial de Tecnologías

- Oracle. *Java SE Documentation*. Oracle, 2025. <https://docs.oracle.com/en/java/>
- Spring. *Spring Boot Reference Documentation*. Pivotal, 2025. <https://spring.io/projects/spring-boot>
- PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2025. <https://www.postgresql.org/docs/>
- Mozilla Developer Network. *HTML, CSS y JavaScript*. Mozilla, 2025. <https://developer.mozilla.org>

Libros y Manuales

- Deitel, P., & Deitel, H. *Java: How to Program*. Pearson, 2023.
- Freeman, E., & Robson, E. *Head First HTML and CSS*. O'Reilly, 2020.

Artículos y Recursos Web

- Booksy. *Gestión de reservas online*. Booksy, 2025. <https://www.booksy.com/es>
- Fresha. *Software de gestión de salones*. Fresha, 2025. <https://www.fresha.com>
- Koibox. *Plataforma de reservas para peluquerías*. Koibox, 2025. <https://www.koibox.com>

Herramientas y Software Utilizado

- IntelliJ IDEA. JetBrains, 2025. <https://www.jetbrains.com/idea/>
- Visual Studio Code. Microsoft, 2025. <https://code.visualstudio.com/>
- Git. Software de control de versiones, 2025. <https://git-scm.com/>
- GitHub. Plataforma de repositorios, 2025. <https://github.com/>
- **ChatGPT**. OpenAI, 2025. <https://openai.com/chatgpt>
- **Gemini**. Google, 2025. <https://gemini.google.com/>